

Report

0.1

Customer

Epineon



NAPQES — AEAD Scheme

July 5, 2026

Contents

1. Document Revisions	3
2. Introduction	3
3. System Overview	4
4. Methodology	5
5. Findings	7
5.1. Critical	8
5.2. Major	11
5.3. Moderate.....	27
5.4. Minor.....	34
6. Conclusion	41
About Us	43
Contacts	43

1. Document Revisions

#	Date	Author	Description
0.1	July 5, 2026	Dmitry Khovratovich	Initial report draft.

2. Introduction

ALL MODIFICATIONS TO THIS DOCUMENT ARE PROHIBITED. VIOLATORS WILL BE PROSECUTED TO THE FULL EXTENT OF THE U.S. LAW.

Client Comment

IT IS IMPORTANT TO NOTE THAT A SECURITY AUDIT IS NOT A GUARANTEE OF ABSOLUTE SECURITY BUT RATHER A SNAPSHOT OF THE SYSTEM'S SECURITY POSTURE AT A SPECIFIC POINT IN TIME. WHILE WE STRIVE TO UNCOVER ALL POTENTIAL ISSUES, THE EVOLVING NATURE OF BLOCKCHAIN TECHNOLOGY AND DEFI MEANS NEW VULNERABILITIES CAN EMERGE.

3. System Overview

NAPQES (Noise-Augmented Post-Quantum Encryption System) is an Authenticated Encryption with Associated Data (AEAD) scheme whose entire security is claimed to reduce to the pseudorandomness of HMAC-SHA256, avoiding the finite-field and permutation structures used by AES-GCM and ChaCha20-Poly1305. A NAPQES key is an ordered tuple $\mathbf{k} = (k_1, \dots, k_K)$ of K distinct primes drawn uniformly from the set $\mathcal{P}$ of primes in $[10^6, 9.9 \times 10^6]$, serialised as the concatenation of five-byte big-endian encodings. All internal values are produced by a domain-separated derivation $\text{Derive}_d(\text{kb}, N, \text{ctx}) = \text{HMAC}(\text{kb}, N \| d \| \text{ctx})$, where the one-byte domain separator d selects among the noise-position, real-token addend, noise probability, authentication tag, noise character, noise-token addend, padding codepoint, and keystream functions (domains $0x00$ – $0x07$), with two further domains ($0x08$ – $0x09$) reserved for the streaming format.

Encryption proceeds by first padding the plaintext to a power-of-two length bucket $B = \max(16, 2^{\lceil \log_2(n+1) \rceil})$, prepending a two-byte length field and appending HMAC-derived padding codepoints from domain $0x06$. The padded codepoints are then encoded as a stream of prime-indexed tokens: each real codepoint c is emitted as $c \cdot k + a$ for the active prime k and a derived addend $a \in [1, k - 1]$, and pseudorandomly interleaved noise tokens $\text{noise_c} \cdot k + \text{noise_a}$ are inserted according to a per-position noise probability $p \in [0.75, 0.99]$. The token blob is LEB128-encoded, XOR-masked with a keystream derived under domain $0x07$, and framed into the version-6 wire format $C = N \| \widetilde{B} \| T$, where N is a 16-byte nonce and $T = \text{Derive}_{0x03}(\text{kb}, N \| \widetilde{B})$ is a 32-byte encrypt-then-MAC authentication tag binding the associated data.

To address release of unverified plaintext in the basic streaming mode, the paper introduces an online-AE streaming format (`encrypt_stream_ae/decrypt_stream_ae`) that splits the masked blob into chunks of at most 1024 bytes, attaches a per-chunk tag T_i under domain $0x08$, and terminates the stream with a sentinel tag T_{final} under domain $0x09$ that binds the total chunk count; the decryptor verifies each chunk tag before releasing plaintext and rejects reordering and truncation.

The security analysis argues IND-CPA confidentiality and EUF-CMA/INT-CTXT integrity under the PRF assumption on HMAC-SHA256, composed into an IND-CCA claim, and discusses the post-quantum security level under Grover search. This report assesses the scheme as described in the ePrint preprint, focusing on the correctness of these definitions and reductions, the confidentiality implications of the wire format and padding scheme, the domain-separation argument, and the completeness and consistency of the specification.

/docs/

napeq-eprint-preprint.tex

/rust/src/

kem.rs

lib.rs

main.rs

self_test.rs

Client Comment

4. Methodology

The methodology utilized by ABDK Consulting is a combination of established audit tactics and methods specifically tuned for the requirements and technological stack of the NAPQES — AEAD Scheme and Rust Implementation Audit project. This approach is flexible and adapted based on the specific project structure, technologies used, and the client's expectations for the security review.

Our audit process incorporates several distinct phases of analysis:

General Code Assessment

The codebase is first reviewed for clarity, consistency, style, and adherence to established best practices. This phase includes checking indentation standards, naming conventions (e.g., following a style guide), appropriate commenting, code duplication, and confusing or irrelevant naming. At this phase, we establish an understanding of the overall system structure and codebase organization.

Entity Usage Analysis

Usages of various entities (contracts, libraries, functions, state variables) defined in the code are analyzed. This includes both internal usages from other parts of the system as well as potential external interactions. We verify that entities are defined with appropriate visibility scopes (internal, external, public, private) and access levels. At this phase, we understand the overall system architecture and how different parts of the code are related and interact with external interfaces.

Access Control Analysis

For those entities that can be accessed externally or manage sensitive operations (like data submission or role management), access control measures are rigorously analyzed. We check that access control mechanisms are correctly implemented, relevant to the intended function, and robust against circumvention. At this phase, we understand the defined user roles, permissions logic, and what digital assets the system ought to protect.

Code Logic Analysis

The specific code logic of critical functions is analyzed for correctness, efficiency, and security. We verify that the code actually performs its intended function as described in the system overview, that algorithms are optimal for on-chain execution, and that proper data types are used to prevent overflow/underflow issues. We also ensure that external libraries used are up to date and relevant. At this phase, we understand the data structures used and their specific purposes.

We classify all identified issues uncovered during these phases by the following severity levels:

Critical

Directly affects the smart contract functionality and may cause a significant loss of assets (e.g., permanent lock of funds, unauthorized minting, total system compromise) or a complete failure of the platform's core invariants.

Major

Is either a solid performance problem or a sign of significant misuse potential: a slight code modification or environment change may lead to loss of funds or data integrity issues. Sometimes it is an abuse of unclear code behavior which should be double-checked by the client for specific edge-case business logic.

Moderate

Is not an immediate functional problem but rather suboptimal performance in edge cases, an obviously bad code practice, or a situation where the code is correct only in certain expected business flows (lacking robust handling of all possible inputs).

Minor

Covers code style improvements, adherence to best practices that do not immediately present a security risk, and general suggestions for code clarity, gas efficiency, or future-proofing the system.

5. Findings

We have completed our technical audit of the NAPQES — AEAD Scheme and Rust Implementation Audit codebase and provided the client with a detailed list of recommendations and identified vulnerabilities. The following statistics summarize the issues discovered and their current resolution status at the time of this report's finalization.



MinorFound
8Fixed
0

Fixed 0 out of 32 issues reported.

5.1. Critical

CVF-1: open

Category: **flaw**

Ciphertext byte-length leaks plaintext content — the IND-CPA theorem is false as proved

A real token is emitted as $token = c \cdot k + a$ with addend $a \in [1, k - 1]$ and prime $k \approx 10^6\text{--}10^7 (\approx 2^{20}\text{--}2^{23})$. The token vector is serialised with unsigned LEB128 (variable length) and only then XOR-masked with the domain-`\texttt{0x07}` keystream. Because the LEB128 byte-length of a word grows with the magnitude of $c \cdot k$, the ciphertext length grows with the codepoint value: $c = 1$ gives a token $\approx 2^{20}$ (~3 LEB128 bytes) while $c = 65535$ gives $\approx 2^{36}$ (~6 bytes). Two plaintexts of the same codepoint length — hence the same padding bucket, the same deterministic noise pattern, identical noise tokens and addends — therefore produce ciphertexts of different byte-length. The mask is length-preserving (truncated to the blob length), so it does not hide this.

Distinguisher: encrypt $m_0 =$ (n copies of U+0001) versus $m_1 =$ (n copies of U+FFFF); m_1 's ciphertext is strictly longer with overwhelming probability, giving IND-CPA advantage ≈ 1 . This holds even under fresh nonces, since the mean ciphertext length over repeated encryptions is content-dependent.

The IND-CPA proof (Theorem 1, hiding lemma) is wrong at the step "since B is the same length for both plaintexts m_0, m_1 (they have equal padded lengths by the IND-CPA requirement)": equal padded codepoint count does not imply equal byte-length, because the token magnitude — and hence its LEB128 length — depends on the plaintext value. The OTP argument only establishes uniformity of the masked blob conditioned on its length, and that length itself carries the challenge bit.

This is not implementation-specific: any faithful implementation using variable-magnitude tokens plus LEB128 with no length normalisation exhibits the leak.

Recommendation

Make the serialised, pre-mask token blob length independent of plaintext content. For example, encode each token in a fixed-width field (a constant number of bytes

per token, sized for the maximum $c \cdot k + a$), or otherwise pad the blob to a length that depends only on the padded codepoint count. Correspondingly fix the hiding lemma: either prove that the masked-blob length is a function of the padded length alone, or drop the false “equal padded length implies equal byte length” step and re-derive confidentiality against the actual, content-dependent length distribution.

Client Comment

/docs/napseq-eprint-preprint.tex

```

161 \item  $\mathrm{varint}(x)$  denotes unsigned LEB128 encoding of
    non-negative integer  $x$ .
243 emit c * k + a
259 where  $N$  is a 16-byte random nonce,  $\widetilde{B}$  is the
    domain-\texttt{0x07}
260 XOR-masked LEB128 token blob, and
476 and since  $B$  is the same length for both plaintexts  $m_0, m_1$  (they
    have
477 equal padded lengths by the IND-CPA requirement),  $\tilde{B}$  is
    uniformly
478 distributed independently of the plaintext bit- $b$  and the view- $V$ .

```

/rust/src/lib.rs

```

275 cypher.push(c as u64 * k + addend);
336 fn b128_encode_tokens(tokens: &[u64]) -> Vec<u8> {
402 cypher.push(c as u64 * k + addend);
411 let masked: Vec<u8> = blob.iter().zip(ks.iter()).map(|(a, b)| a ^
    b).collect();

```

CVF-2: open

Category: **behavior**

Domain-separation layout is inconsistent between Table 1, the security proofs, and the reference code

Three independent descriptions of the per-domain key-derivation input disagree, and no single layout is used consistently.

1. Table 1 defines a uniform nonce-first layout $\text{Derive}_d(\text{kb}, N, \text{ctx}) = \text{HMAC}(\text{kb}, N \| d \| \text{ctx})$ for all ten domains, placing the nonce N first and the domain byte d second.

2. The `rem:domsep` remark and the “OTP argument” split the domains into two groups and reason about them as if the layout depended on the group, rather than being a single injective encoding. This produces a probabilistic cross-group collision term instead of an unconditional separation argument.
3. The reference code does not follow Table 1 uniformly: domain `0x02` is nonce-first (`N \|| 0x02`, lib.rs 178–179), whereas domains `0x03`, `0x08`, `0x09` are domain-first (`0x03 \|| ...`, `0x08 \|| ...`, `0x09 \|| ...`, lib.rs 185–187). The wire-format tag $T = \text{Derive}_{0x03}(\text{kb}, N \parallel \widetilde{B})$, read through the Table 1 formula, is nonce-first and omits the AAD, while the INT-CTXT and AAD-binding sections use the domain-first, AAD-bound input $T = \text{HMAC}(\text{kb}, 0x03 \parallel \text{be4}(|A|) \parallel A \parallel N \parallel \widetilde{B})$.

The specification, the proofs, and the implementation therefore describe mutually incompatible byte layouts for the HMAC input. Because domain separation is the foundation of every confidentiality and integrity claim in the scheme, this inconsistency is not merely presentational: depending on which layout is actually deployed, the AAD may be unauthenticated, cross-domain inputs may collide, and the proofs cease to apply to the running code.

Recommendation

Adopt a single uniform domain-first layout $\text{HMAC}(\text{kb}, d \parallel N \parallel \text{ctx})$ for all ten domains, and length-prefix every variable-width field (e.g. $\text{be4}(|A|) \parallel A$ for AAD). With this layout, prove domain separation unconditionally rather than probabilistically:

- Cross-domain inputs differ in byte 0 (the domain tag d), so they can never collide.
- Intra-domain inputs place N at a fixed offset and follow it with fixed-width counters or length-prefixed AAD, making the encoding injective.

Rewrite the “OTP argument” paragraph around this injectivity so that no probabilistic cross-group term is required at all. Update Table 1, the `rem:domsep` remark, the wire-format tag definition, and the reference code (lib.rs domains `0x02`, `0x03`, `0x08`, `0x09`) so all three artifacts use the identical domain-first, length-prefixed input, and confirm the AAD is authenticated.

Cross-reference: the probabilistic cross-group collision term whose removal this fix enables is the subject of issue #848; unifying to a domain-first injective layout makes that term vanish unconditionally.

Client Comment

/docs/napseq-eprint-preprint.tex

```

184   \mathrm{Derive}_d(\mathsf{kb}, N, \mathsf{ctx}) =
      \mathrm{HMAC}(\mathsf{kb}, N \parallel d \parallel \mathsf{ctx}),
196   \texttt{0x02} & \text{Noise probability} & \text{(none)} & \$[0.75, 0.99]\$ \\\
197   \texttt{0x03} & \text{Auth tag (block)} & \$\mathrm{be4}(\lvert\text{aad}\rvert) & \text{aad} \parallel \mathsf{payload} \$ & 32 \text{ bytes} \\\

```

```

202 \texttt{0x08} & Chunk tag (stream AE) & $\mathrm{be4}(\lvert\mathrm{aad}
\mathrm{rvert}) \mathrm{||} \mathrm{aad} \mathrm{||} \mathrm{N} \mathrm{||} \mathrm{be4}(i) \mathrm{||} \mathrm{tilde{C}}_i$ & 32 bytes \
203 \texttt{0x09} & Final sentinel tag (stream AE) & $\mathrm{be4}(\lvert\mathrm{aad}
\mathrm{rvert}) \mathrm{||} \mathrm{aad} \mathrm{||} \mathrm{N} \mathrm{||} \mathrm{be4}(n_{\mathrm{chunks}}))$ & 32
bytes \
261 $T = \mathrm{Derive}_{\mathrm{0x03}}(\mathrm{ksf}\{kb\}, \mathrm{N} \mathrm{||} \mathrm{widetilde{B}})$
306 domain $\mathrm{0x03}$ (authentication tag) uses input
307 $\mathrm{0x03} \mathrm{||} \mathrm{be4}(|A|) \mathrm{||} A \mathrm{||} \mathrm{N} \mathrm{||} \mathrm{widetilde{B}}$
308 (variable length, at minimum $1{+}4{+}16 = 21$, B with empty
AAD and empty blob).
309 \end{itemize}
453 \item Noise probability: $\rho(\texttt{0x02} \mathrm{||} \mathrm{N}^*)$ (domain-first,
17\,B)
484 (domain-$\texttt{0x03}$ is domain-first; see Table~\ref{tab:domains}).
548 $\texttt{0x03} \mathrm{||} \mathrm{be4}(|\mathrm{aad}^*|) \mathrm{||} \mathrm{aad}^* \mathrm{||} \mathrm{N}^* \mathrm{||}
\mathrm{widetilde{B}}^*\mathrm{right})$.
549 \]
584 $T = \mathrm{HMAC}(\mathrm{ksf}\{kb\}, \texttt{0x03} \mathrm{||} \mathrm{be4}(|A|)
\mathrm{||} A \mathrm{||} \mathrm{N} \mathrm{||} B)$.

```

/rust/src/lib.rs

```

178 buf.extend_from_slice(nonce);
179 buf.push(0x02);
185 fn compute_auth_tag(kb: &[u8], aad: &[u8], payload: &[u8]) -> [u8;
32] {
186     let mut buf = Vec::with_capacity(1 + 4 + aad.len() +
payload.len());
187     buf.push(0x03);

```

5.2. Major

CVF-3: open

Category: **flaw**

Nonce reuse yields key recovery via the length channel — contradicting the Table 4 “No” claim

Every internal value — noise positions (`\texttt{0x00}`), noise characters (`\texttt{0x04}`), addends (`\texttt{0x01}`, `\texttt{0x05}`), padding (`\texttt{0x06}`) and the keystream (`\texttt{0x07}`)

— is a deterministic function of (kb, N) only, via $\text{Derive}_d(kb, N, \text{ctx}) = \text{HMAC}(kb, N \| d \| \text{ctx})$; nothing else is random. A repeated nonce therefore reproduces an identical keystream (a two-time pad) and identical noise/addends. Combined with the ciphertext-length channel, this escalates to full key recovery: under a fixed nonce, run CPA varying the codepoint c at a chosen real position j ; everything is constant except $\text{token}_j = c \cdot k_j + a_j$. As c increases, the token crosses a LEB128 length boundary at $c \cdot k_j + a_j \approx 2^{7m}$, and the attacker detects the transition codepoint c^* from the ciphertext length, yielding $k_j \approx 2^{7m}/c^*$. Binary-searching the transition recovers each of the K primes without ever attacking the mask.

For an ordinary stream cipher, nonce reuse loses confidentiality but not the key; here the affine $c \mapsto c \cdot k + a$ structure makes reuse strictly worse. Table 4 nonetheless advertises "Nonce-reuse key recovery: No" as an advantage over AES-GCM and ChaCha20-Poly1305, which is false for this construction. The 128-bit random nonce makes accidental reuse a $\approx 2^{64}$ birthday event, but the misuse resistance the paper claims is absent, and the caller-chosen-nonce entry point compounds the hazard.

Recommendation

Correct the Table 4 claim to state that nonce reuse is catastrophic (key-recoverable) for NAPQES. More fundamentally, remove the ciphertext-length side channel (see the length-leak finding) so a repeated nonce cannot be leveraged into prime recovery, and adopt a misuse-resistant or synthetic-IV design (deriving the nonce/IV from the message) if misuse resistance is a design goal. Restrict or remove caller-chosen-nonce entry points outside testing.

Client Comment

/docs/napseq-eprint-preprint.tex

```
184 \mathrm{Derive}_d(\mathsf{kb}, N, \mathsf{ctx}) =
    \mathrm{HMAC}(\mathsf{kb}, N \| d \| \mathsf{ctx}),
736 Nonce-reuse key recovery & \textbf{No} & Yes (GCM key) & Yes
    (Poly1305 key) & Partial \\\
```

/rust/src/lib.rs

```
252 pub fn encrypt(message: &[u32], key: &[u64]) -> ([u8; NONCE_SIZE],
    Vec<u64>) {
378 pub fn encrypt_bytes(message: &str, key: &[u64], aad: &[u8]) ->
    Result<Vec<u8>, String> {
424 pub fn encrypt_bytes_with_nonce(
```

CVF-4: openCategory: **architecture**

All confidentiality rests on the 0x07 keystream; the prime-token layer is decorative and only creates the length leak

Because the wire value is $\widetilde{B} = \text{LEB128}(\text{tokens}) \oplus \text{keystream}_{0x07}$, the tokens never appear in the clear, so confidentiality reduces entirely to domain `\texttt{0x07}` being a good HMAC-CTR stream cipher. The prime multiplication and noise injection contribute zero confidentiality — they are masked away — while expanding length and creating the content-dependent length leak reported separately. This is reflected in the hiding lemma itself, whose OTP argument only ever uses the `\texttt{0x07}` keystream to establish confidentiality; the token structure plays no role in the confidentiality proof.

Consequences: the advertised $\approx 2^{196}$ prime-tuple key space is illusory for confidentiality; if the mask is ever weak or reused, the exposed tokens are trivially invertible to the key (see the length-leak and nonce-reuse findings). The security narrative — “prime-indexed token cipher”, “quantum-resistant algebra-free design” — describes a layer that does no confidentiality work: the scheme is effectively HMAC-CTR plus an HMAC tag, with an expensive and leaky obfuscation layer on top.

Recommendation

Either (i) drop the prime/noise obfuscation layer and present NAPQES honestly as an HMAC-CTR-based AEAD, basing all security claims on the `\texttt{0x07}` keystream and the HMAC tag; or (ii) if the prime layer is intended to provide security, redesign it so it is not simply XOR-masked away, and prove a confidentiality property that actually depends on it. In either case, align the security narrative and the 2^{196} key-space claim with the layer that does the real work.

Client Comment**/docs/napseq-eprint-preprint.tex**

```
259 where $N$ is a 16-byte random nonce,  $\widetilde{B}$  is the
    domain-\texttt{0x07}
260 XOR-masked LEB128 token blob, and
445 \noindent\textbf{Independence of  $\mathrm{ks}(N^*)$  from  $B$  (OTP
    argument).}
469 Consequently, the one-time-pad identity applies: for any fixed
    deterministic  $B$ ,
```

/rust/src/lib.rs

```
229 fn varint_keystream(kb: &[u8], nonce: &[u8], length: usize) ->
    Vec<u8> {
411     let masked: Vec<u8> = blob.iter().zip(ks.iter()).map(|(a, b)| a ^
        b).collect();
```

CVF-5: openCategory: **architecture**

Construction lacks the formal AEAD algorithm triple, so the scheme is not well defined

The Construction section describes NAPQES only informally — notation, key, domain-separated derivation, padding, the token-emission loop, and the wire format — but never states the scheme as the formal algorithm triple that an AEAD definition requires: a key-generation/setup algorithm, an encryption algorithm $\text{Enc}(K, N, A, M) \rightarrow C$, and a decryption algorithm $\text{Dec}(K, N, A, C) \rightarrow M$ or $\perp$, each with explicitly typed inputs and outputs and a stated correctness condition. Standard AEAD specifications define these precisely (cf. the Ascon v1.2 submission, <https://competitions.cr.yp.to/round3/asconv12.pdf>). Without them the scheme is not well defined: the exact inputs (key, nonce, associated data, plaintext), the decryption and verification procedure together with its authentication-failure behaviour, and the correctness/soundness statements that the security theorems quantify over are all left implicit and scattered across prose and pseudocode. This blocks both independent implementation and rigorous security analysis, and it is the root cause of several of the other findings (undefined wire-format value, missing derivation formulas, undefined encodings).

Recommendation

Add a formal specification of NAPQES as an AEAD triple — KeyGen/Setup, Encrypt, and Decrypt — with fully typed inputs and outputs, an explicit correctness condition ($\text{Dec}(K, N, A, \text{Enc}(K, N, A, M)) = M$) and an explicit authentication-failure output ($\perp$), following the structure of a standard AEAD definition such as the Ascon v1.2 submission. State the security goals (IND-CPA / IND-CCA and EUF-CMA integrity) against these formally defined algorithms, so that every subsequent theorem and construction detail refers to a single well-defined object.

Client Comment

/docs/napseq-eprint-preprint.tex

149 `\section{Construction}`

CVF-6: openCategory: **documentation**

Token emission loop pseudocode is incomplete: its derivation primitives are undefined

The token-emission pseudocode invokes helper primitives — `\texttt{is_noise}` (domain `\texttt{0x00}`), `\texttt{derive_noise_char}` (`\texttt{0x04}`), `\texttt{derive_noise_addend}` (`\texttt{0x05}`), and `\texttt{derive_real_addend}` (`\texttt{0x01}`) — that are never formally defined. Table

1 lists each domain's context input and output range, but the concrete derivation formulas that map the 32-byte $\text{Derive}_d(\text{kb}, N, \text{ctx})$ output to the required value — a Boolean noise decision, a character in $[32, 127]$, an addend in $[1, k - 1]$ — are not given. As a result the pseudocode cannot be executed as written and the token construction is not reproducible from the paper: the exact reduction of the HMAC output into each range, and hence the potential for modulo bias, is unspecified for every one of these primitives.

Recommendation

Give an explicit defining formula for each helper primitive used in the loop, mapping $\text{Derive}_d(\text{kb}, N, \text{ctx})$ to its output for domains `\texttt{0x00}`, `\texttt{0x01}`, `\texttt{0x04}`, and `\texttt{0x05}`: state which bytes are consumed and the exact reduction into the target range, addressing modulo bias for the finite ranges. Either replace the named functions in the pseudocode with these formulas or add a definitions subsection that the pseudocode references, so the token-emission loop is fully specified and reproducible.

Client Comment

/docs/napseq-eprint-preprint.tex

```

234 while is_noise(ct_pos):           # domain 0x00
236     noise_c = derive_noise_char(ct_pos) # domain 0x04; in
      [32,127]
237     noise_a = derive_noise_addend(ct_pos, k) # domain 0x05; in
      [1,k-1]
242 a = derive_real_addend(real_idx, k) # domain 0x01; in [1,k-1]
```

CVF-7: open

Category: **documentation**

Applicability of the Online-AE streaming format relative to the other formats is unspecified

The paper defines at least three encodings — the block-mode Wire Format (Version 6) $C = N \parallel \widetilde{B} \parallel T$, a "basic streaming format" (`\texttt{encrypt_stream}`, single trailing tag), and the online-AE streaming format (v6s-ae, per-chunk tags) — but never states their relationship or when each applies. It is unclear whether the online-AE streaming format is the sole normative format, one of several interchangeable options, or a mode selected under specific conditions; whether the block-mode and streaming formats are meant to coexist; and how a decryptor determines which format a received ciphertext uses, since

the wire layout carries no version/format discriminator. This ambiguity has security consequences: the basic streaming format is stated to release unverified plaintext (RUP, formerly CAV-001), so if it remains a permitted format a receiver could be induced to run the unsafe decoder. Without a stated format-selection rule the scheme is not operationally well defined.

Recommendation

State explicitly which format(s) are normative and under what conditions each is used, whether the block-mode and streaming formats coexist, and — if more than one is permitted — how the format/version is encoded in the ciphertext and authenticated so that a decryptor selects the correct and safe decoder unambiguously. If the RUP-prone basic streaming format is superseded by the online-AE streaming format, mark it deprecated and forbid its use.

Client Comment

/docs/napseq-eprint-preprint.tex

```

265 \subsection{Online-AE Streaming Format (v6s-ae)}
268 The basic streaming format (\texttt{encrypt\_stream}) carries a
    single tag at the
269 end of the stream, causing \texttt{decrypt\_stream} to release
    plaintext before
270 the tag is verified (RUP, formerly CAV-001). The online-AE streaming
    format
271 eliminates this by attaching a per-chunk HMAC-SHA256 tag to each
    segment of the
272 masked varint blob.
```

CVF-8: open

Category: **algorithm**

IND-CPA bound ignores the actual key entropy and key size

Theorem 1 (IND-CPA) bounds the adversary's advantage by $\text{Adv}_{\text{HMAC-SHA256}}^{\text{PRF}}(\mathcal{B}_1) + q^2/2^{128}$, with no term depending on the key. This is not a sound reduction, because the HMAC key is not a uniform bit-string: it is $\text{kb} = \text{key_bytes}(\mathbf{k})$, the serialisation of an ordered tuple of K distinct primes from $\mathcal{P}$, whose min-entropy is bounded by $\log_2(|\mathcal{P}|!/(|\mathcal{P}| - K)!)$ — the paper's own $\approx 2^{196}$ for $K = 10$. The standard HMAC-SHA256 PRF assumption is stated for a uniformly random key of a given length; the PRF advantage against HMAC

keyed by this low-entropy, structured distribution is a different quantity and includes an exhaustive-key-search term $\approx q_F \cdot (|\mathcal{P}|!/(|\mathcal{P}| - K)!)^{-1}$, where q_F is the number of offline HMAC evaluations the adversary can perform.

Consequences:

- The bound is independent of K and $|\mathcal{P}|$, so it does not degrade as the key space shrinks. For small K (e.g. $K = 1$, key space $\approx 2^{20}$) the key is brute-forceable: the true IND-CPA advantage approaches 1 after $\approx 2^{20}$ offline HMAC evaluations, yet the stated bound would still be negligible — so the theorem as literally written is false for admissible small key sizes.
- Even for $K = 10$ the reduction is unsound as written: it silently substitutes "HMAC is a PRF under a uniform key" for "HMAC is a PRF under a key drawn from the prime-tuple distribution," which are different assumptions with different advantages. The $\approx 2^{196}$ key-space figure never enters the bound.
- Key length also interacts with HMAC key processing: for $5K > 64$ bytes the key is hashed to 256 bits before use and shorter keys are zero-padded to the block size, so the map from the entropy source to the effective HMAC key is not addressed by the analysis.

Recommendation

Restate the bound to include the actual key entropy: add a key-guessing term $q_F \cdot 2^{-H_\infty(\mathbf{k})}$ with $H_\infty(\mathbf{k}) = \log_2(|\mathcal{P}|!/(|\mathcal{P}| - K)!)$, and specify the minimum K (and $|\mathcal{P}|$) required to meet the target security level, so the theorem holds for all admissible key sizes and no longer overstates security for small K . Make the PRF assumption's key distribution match the actual key_bytes distribution, or key HMAC with a uniformly derived subkey (e.g. via a KDF over kb) so that the standard uniform-key PRF assumption legitimately applies, and state the HMAC key-length handling explicitly.

Client Comment

/docs/napseq-eprint-preprint.tex

```

175 ordered tuple provides a key space of:
337 \begin{theorem}[IND-CPA]\label{thm:ind-cpa}
338   Under the PRF assumption on HMAC-SHA256, the NAPQES block
      encryption scheme
339   is IND-CPA secure. Specifically, for any PPT adversary
      $\mathcal{A}$ making
340   at most $q$ encryption queries there exists a PRF adversary
      $\mathcal{B}_1$
341   with similar running time such that:

```

```

343   \mathrm{Adv}^{\mathrm{IND}\text{-}}
      \mathrm{CPA}}_{\mathrm{NAPQES}}(\mathrm{A})
344   \leq
345   \mathrm{Adv}^{\mathrm{PRF}}_{\mathrm{HMAC}\text{-}}
      \mathrm{SHA256}}(\mathrm{B}_1)
346   + \frac{q^2}{2^{128}}.
356 of distinct primes in~ $\mathcal{P}$ , computes
       $\mathsf{key\_bytes}(\mathbf{k})$ ,
357 and answers up to  $q$  chosen-plaintext queries honestly using the real

```

CVF-9: openCategory: **architecture**

IND-CPA is never defined and the challenge-equality constraint is ill-specified, so Theorem 1 is not well-defined

The paper claims the scheme “is IND-CPA secure” (Theorem 1) and proves it via “the standard IND-CPA experiment”, but no formal IND-CPA definition appears anywhere: there is no definition of the security notion, its adversary, its oracles, or its advantage — the experiment is only sketched inside the proof. The challenge constraint is then stated as “ $\mathcal{A}$ submits a challenge pair (m_0, m_1) of equal padded length”, which is both non-standard and ambiguous:

- Standard IND-CPA requires $|m_0| = |m_1|$ (equal plaintext length) and imposes no “padded length” restriction; introducing an “equal padded length” precondition silently narrows the admissible adversaries.
- The statement never says how length is measured — codepoints, bytes, or the padded bucket B — and these genuinely differ (B is a power-of-two function of the codepoint count, whereas the on-wire byte length depends on token magnitude).

Because the length metric is undefined, the theorem statement is not well-defined, and the proof does not genuinely conclude: it later invokes “equal padded lengths by the IND-CPA requirement” in the hiding lemma to assert that the masked blob has equal length for m_0 and m_1 — exactly the circular step, where the “requirement” is smuggled into an unstated definition to make the proof close. Under the correct notion (equal plaintext length, no padded-length restriction) the scheme is not IND-CPA secure (see the ciphertext-length-leak finding); under the paper’s bespoke “equal padded length” notion the theorem establishes something strictly weaker than IND-CPA while still labelling it “IND-CPA secure”. Either way the stated security claim is not established.

Recommendation

Add a formal IND-CPA definition (adversary, encryption oracle, challenge, advantage) before Theorem 1, using the standard constraint $|m_0| = |m_1|$ with the length metric stated explicitly and matching what the adversary actually observes (on-wire byte length). Do not impose an “equal padded length” precondition; if the scheme only

achieves security for equal-padded-length challenges, define that restricted notion explicitly and do not call it IND-CPA. Then re-prove — or, per the length-leak finding, refute — the theorem against the properly defined notion, noting that standard IND-CPA cannot hold until the ciphertext-length side channel is removed.

Client Comment

/docs/napseq-eprint-preprint.tex

```

339   is IND-CPA secure. Specifically, for any PPT adversary
       $\mathcal{A}$  making
354 This is the standard IND-CPA experiment. The challenger samples
361  $\mathcal{A}$  submits a challenge pair  $(m_0, m_1)$  of equal padded
      length;
477 equal padded lengths by the IND-CPA requirement),  $\tilde{B}$  is
      uniformly

```

CVF-10: open

Category: **algorithm**

INT-CTXT bound omits the ideal-world tag-guessing term and is false for $q = 0$

The INT-CTXT proof reduces a forgery to distinguishing HMAC from a random function, but never accounts for the fact that even against a truly random function the adversary forges by guessing T^* with probability 2^{-256} . The reduction $\mathcal{B}_2$'s advantage is $\Pr[\text{forge} \mid \text{real}] - \Pr[\text{forge} \mid \text{random}]$, so the correct bound must carry the ideal-world term: $\text{Adv}^{\text{INT-CTXT}} \leq \text{Adv}^{\text{PRF}}(\mathcal{B}_2) + q^2/2^{128} + 2^{-256}$. As stated (without 2^{-256}) the bound is strictly false for $q = 0$: an adversary that makes no queries and submits a random tag still forges with probability $2^{-256} > 0$, exceeding the claimed bound of 0. The proof also never states $\mathcal{B}_2$'s decision rule (output "real" iff the forgery verifies), so the advantage accounting is entirely absent.

The same omission propagates to the IND-CCA bound, which invokes Theorem 2 with q_d verification queries and should therefore carry a $q_d/2^{256}$ term that is likewise absent from the composed bound.

Recommendation

Add the ideal-world forgery term to the bound (2^{-256} for a single verification attempt, or $q_v/2^{256}$ for q_v verification queries), and state $\mathcal{B}_2$'s distinguishing rule explicitly (guess "real" iff $\mathcal{A}$'s forgery verifies), deriving its advantage as the gap between the real-world and ideal-world forgery probabilities. Carry the $q_d/2^{256}$ form through into the IND-CCA bound.

Client Comment**/docs/napseq-eprint-preprint.tex**

```

531     \mathrm{Adv}^{\mathrm{INT}\text{-}
      }CTXT}_{\mathrm{NAPQES}}(\mathrmcal{A})
532     \leq
533     \mathrm{Adv}^{\mathrm{PRF}}_{\mathrm{HMAC}\text{-}
      }SHA256}_{\mathrmcal{B}_2}
534     + \frac{q^2}{2^{128}}.
560 from a random function.
561
656     + \frac{(q+q_d)^2}{2^{128}},

```

CVF-11: openCategory: **algorithm**

INT-CTXT proof's nonce-collision probability is bogus — N^* is adversarially chosen

The proof asserts "the probability of a nonce collision satisfying $N^* = N_i$ for some i is at most $q/2^{128} \leq q^2/2^{128}$ by the union bound." This is a category error: N^* is part of the forgery c^* and is chosen by $\mathcal{A}$ after observing $N_1, \dots, N_q$ in the returned ciphertexts. The adversary can set $N^* = N_i$ with probability 1; it is not a random event and cannot be union-bounded. Moreover Case 2 already handles $N^* = N_i$ deterministically (arguing that sub-case cannot yield a valid fresh forgery), so no probability term is needed there at all. Consequently the $q^2/2^{128}$ term appearing in the theorem statement is not derived from anything coherent in this proof.

Recommendation

Remove the spurious "nonce collision" probability argument. Handle $N^* = N_i$ deterministically via injectivity of the length-prefixed tag encoding (as Case 2 already attempts), and re-derive the theorem bound so that every term traces to an actual step. Any residual q -dependent term must come from a genuine collision event (e.g. the cross-group HMAC-input collision), not from a union bound over an adversarially chosen nonce.

Client Comment

/docs/napseq-eprint-preprint.tex

```

534 + \frac{q^2}{2^{128}}.
570 The probability of a nonce collision satisfying  $N^* = N_i$  for some
     $i$ 
571 is at most  $q / 2^{128} \leq q^2 / 2^{128}$  by the union bound.
572 Combining both cases:

```

CVF-12: open

Category: **algorithm**

Case 1 freshness justification is wrong, and the cross-group HMAC-input collision term is ignored across INT-CTXT, IND-CPA, and Remark 1

Case 1 claims “all domain-`\texttt{0x03}` tag inputs from the query phase are prefixed by nonces $N_1, \dots, N_q$.” That is false: tag inputs are prefixed by `0x03\|be4(|aadi|)\|aadi`, so the nonce sits at a variable offset depending on $|aad_i|$. Freshness of x^* actually follows from injectivity of the encoding, which is never invoked. More importantly, “ x^* was never submitted to $\text{HMAC}(\text{kb}, \cdot)$ ” must hold against all query-phase HMAC calls, including the nonce-first domains `\texttt{0x00}`–`\texttt{0x07}` used by the encryption oracle, not just the tag calls. A nonce-first input $N_i\|d\|\text{ctx}$ (21–22 bytes) can equal a minimal tag input `0x03\|be4(0)\|N^*` (21 bytes; $|\widetilde{B}^*| = 0$ gives exactly the 48-byte minimum valid ciphertext) whenever N_i begins with `0x03 00 00 00 00` — probability $\approx q/2^{40}$ — a cross-group collision the INT-CTXT proof ignores. Compounding this, Remark 1 (domain separation) claims its $q/2^{32}$ cross-group term “is dominated by the nonce-collision term $q^2/2^{128}$ ”, which is mathematically false for all $q < 2^{96}$ (e.g. $q = 2^{20}$: $q/2^{32} = 2^{-12} \gg 2^{-88} = q^2/2^{128}$). A $q/2^{32}$ -type term must appear explicitly in the bounds.

The IND-CPA hiding lemma repeats the same error from the other direction. The “Freshness of the authentication tag” paragraph states the collision is “at most $1/2^{40}$ per query and absorbed into the existing $q^2/2^{128}$ term.” This is wrong on two counts. First, over q queries the contribution is $\approx q/2^{40}$, and $q/2^{40} \leq q^2/2^{128}$ only for $q \geq 2^{88}$, so the absorption claim fails for every realistic q — the term is not dominated and cannot be silently discarded. Second, the exponent 40 conflates a $1/2^8$ first-byte (`0x03`) match with a 32-bit be4 length-field coincidence; these are distinct events and stating a single $1/2^{40}$ figure obscures the actual per-query probability.

The composed IND-CCA bound inherits this omission: it carries only the $q^2/2^{128}$ and $(q + q_d)^2/2^{128}$ nonce terms and no cross-group collision term.

Recommendation

The clean fix is structural: after unifying to a single domain-first, length-prefixed injective layout (see issue #851), the tag input is unconditionally distinct from every other HMAC input, so the cross-group collision cannot occur and the $1/2^{40} / q/2^{32}$ term vanishes. In that case, delete the "Freshness of the authentication tag" absorption sentence entirely and justify Case 1 freshness by injectivity of the encoding.

If the current mixed layout is retained instead, do not absorb the term: justify Case 1 freshness via injectivity where it holds, account for collisions against all HMAC domains (not only `\texttt{0x03}`), and carry an explicit additive cross-group term (order $q/2^{40}$ in the IND-CPA hiding lemma and INT-CTXT Case 1, $q/2^{32}$ in Remark 1) through the INT-CTXT, IND-CPA, and composed IND-CCA theorem bounds. Correct Remark 1: the $q/2^{32}$ term is not dominated by $q^2/2^{128}$ for realistic q . Also separate the $1/2^8$ first-byte match from the 32-bit length-field coincidence rather than reporting a single conflated $1/2^{40}$.

Client Comment

/docs/napseq-eprint-preprint.tex

```

327 cross-group collision is therefore at most  $q/2^{32}$ , which is
    negligible
328 for  $q \ll 2^{32}$  and is dominated by the nonce-collision term
329  $q^2/2^{128}$  already present in Theorem~\ref{thm:ind-cpa}.
481 \noindent\textbf{Freshness of the authentication tag.}
493  $N^{*}[1{:}4] = \mathrm{be4}(|\mathrm{aad}|)$ , yielding probability at most
     $1/2^{40}$  per
494 query and absorbed into the existing  $q^2/2^{128}$  term.
553 All domain-\texttt{0x03} tag inputs from the query phase are prefixed
    by
554 nonces  $N_1, \ldots, N_q \neq N^*$ , so  $x^*$  was never submitted to
555  $\mathrm{HMAC}(\mathrm{sf}_{\mathrm{kb}}, \cdot)$  during the query phase.
    Producing the
556 correct  $T^*$  is therefore an HMAC MAC forgery at a fresh input. We
682 + \bigl|\Pr[\mathrm{cal}\{A\} \text{ wins } H_1] - \tfrac{1}{2}\bigr|
    \\\
683 &\leq \mathrm{Adv}^{\mathrm{PRF}}_{\mathrm{HMAC}\text{-}
    \mathrm{SHA256}}(\mathrm{cal}\{B\}_2)
684 + \frac{(q+q_d)^2}{2^{128}}
685 + \mathrm{Adv}^{\mathrm{PRF}}_{\mathrm{HMAC}\text{-}
    \mathrm{SHA256}}(\mathrm{cal}\{B\}_1)

```

$$686 \quad + \frac{q^2}{2^{128}}.$$

CVF-13: openCategory: **architecture**

INT-CTXT and IND-CPA reductions cannot simulate the encryption oracle without the prime vector $\mathbf{k}$

The reductions $\mathcal{B}_2$ (INT-CTXT) and $\mathcal{B}_1$ (IND-CPA) “forward all HMAC calls to their own PRF oracle,” but simulating NAPQES encryption also requires the prime vector $\mathbf{k}$ itself: token emission computes $c \cdot k_j + a$ and the addend ranges $[1, k_j - 1]$ depend on k_j . In the PRF game the oracle key is uniform and unknown, whereas in NAPQES $\text{kb} = \text{key_bytes}(\mathbf{k})$ is a deterministic serialisation of those same primes. The reduction therefore does not go through as written: $\mathcal{B}$ cannot extract $\mathbf{k}$ from oracle access, and if it samples its own $\mathbf{k}'$ then the simulated scheme has an HMAC key independent of the primes — a different scheme than NAPQES. Additionally, the HMAC-PRF assumption is for uniform keys, while kb is a structured, non-uniform encoding of primes that is correlated with the arithmetic layer; the proof needs an explicit key-indistinguishability step, or an assumption that HMAC is a PRF under this key distribution and remains one given the correlated use of $\mathbf{k}$. This defect is shared by both the INT-CTXT and IND-CPA proofs (and is related to the separate finding that the IND-CPA bound omits the key-entropy term).

Because the IND-CCA bound is assembled from these two reductions via the Bellare–Namprempre composition, it inherits the same simulation gap: the $\mathcal{B}_1$ and $\mathcal{B}_2$ advantages it invokes are not established for the actual scheme.

Recommendation

Restructure the reductions so the simulator has $\mathbf{k}$ consistently with the PRF oracle key — e.g. key HMAC with a uniformly derived subkey (a KDF over $\mathbf{k}$) so the arithmetic layer and the PRF key are decoupled, letting $\mathcal{B}$ sample $\mathbf{k}$ while forwarding HMAC to the oracle. Otherwise state and justify a PRF assumption for the actual $\text{key_bytes}(\mathbf{k})$ distribution, including its correlation with the token arithmetic, and add the corresponding key-indistinguishability step to the INT-CTXT, IND-CPA, and composed IND-CCA proofs.

Client Comment

/docs/napseq-eprint-preprint.tex

388 $\mathcal{B}_1$ simulates the IND-CPA experiment for $\mathcal{A}$, answering


```

389 every internal HMAC call by forwarding to  $\mathcal{A}$ .
557 construct  $\mathcal{B}_2$  that simulates the INT-CTXT experiment for
558  $\mathcal{A}$ , forwarding all HMAC calls to its own PRF oracle, and
    uses
559  $\mathcal{A}$ 's forgery  $(x^*, T^*)$  to distinguish
     $\text{HMAC}(\text{key}, \cdot)$ 
655  $\text{Adv}^{\text{PRF}}_{\text{HMAC}}(\text{SHA256})_{\mathcal{B}_2}$ 
672 +  $\text{Adv}^{\text{PRF}}_{\text{HMAC}}(\text{SHA256})_{\mathcal{B}_1}$ 

```

CVF-14: openCategory: **architecture**

IND-CCA proof: the "identical-until-bad" step is false — H_1 also diverges on non-fresh replays

The transition $H_0 \rightarrow H_1$ stubs the decryption oracle $\mathcal{D}$ to return $\perp$ always, and claims " H_0 and H_1 are identical in every execution that does not trigger D ", where D requires a fresh $(c \notin \{c_1, \dots, c_q\})$ valid submission. This is false: in H_0 the adversary may legally query $\mathcal{D}(c_i, aad_i)$ on a replayed encryption-oracle output and receive the valid plaintext m_i , whereas H_1 returns $\perp$ — yet that query is not fresh, so D does not occur. An adversary that simply queries $\mathcal{D}(c_1, aad_1)$ distinguishes the two games with probability 1 without ever triggering D , so the fundamental-lemma step $|\Pr[H_0] - \Pr[H_1]| \leq \Pr[D]$ is not established as written.

Recommendation

Define H_1 so that $\mathcal{D}$ returns the recorded plaintext for a replayed (c_i, aad_i) pair and returns $\perp$ only for fresh ciphertexts (the standard hybrid in which the decryption oracle answers known ciphertexts from a table). Then H_0 and H_1 are identical unless D (a fresh valid decryption) occurs, and the identical-until-bad step goes through.

Client Comment**/docs/napseq-eprint-preprint.tex**

```

619 \paragraph{Game  $H_1$  (Decryption oracle stubbed to  $\bot$ ).}
622 \paragraph{Transition  $H_0 \rightarrow H_1$ .}
623 Let  $D$  be the event that  $\mathcal{A}$  submits, during the query
    phase, a

```



```

624 \emph{fresh valid} ciphertext to  $\mathcal{D}$ -i.e., a tuple
625  $(c, \text{aad})$  with  $c \notin \{c_1, \dots, c_q\}$  (the encryption-oracle
626 outputs) such that  $\text{NAPQES}.\text{Dec}(\mathbf{k}, c, \text{aad})$ 
    \neq \bot.
627 The games  $H_0$  and  $H_1$  are identical in every execution that does
    not
    
```

CVF-15: open

Category: **algorithm**

IND-CCA proof: invalid "absorbing the constant" step — $2(q+q_d)^2$ cannot become $(q+q_d)^2$

The combination produces two nonce-collision terms $q^2/2^{128} + (q+q_d)^2/2^{128}$, which the proof bounds by $2(q+q_d)^2/2^{128}$ and then claims that "absorbing the constant into the stated bound yields $(q+q_d)^2/2^{128}$, matching the statement." A multiplicative factor of 2 cannot be absorbed into nothing: $2x \leq x$ is false for $x > 0$. The proof therefore establishes only $2(q+q_d)^2/2^{128}$, so the theorem's stated bound — with a bare $(q+q_d)^2/2^{128}$ — is not proved.

Recommendation

State the honest constant $2(q+q_d)^2/2^{128}$ in both the proof and the theorem, or tighten via $q^2 + (q+q_d)^2 \leq (q+(q+q_d))^2 = (2q+q_d)^2$ and restate the bound accordingly. Do not silently drop the factor of 2.

Client Comment

/docs/napseq-eprint-preprint.tex

```

688 Since  $q^2 + (q+q_d)^2 \leq 2(q+q_d)^2$ , the sum of the two
    nonce-collision
689 terms is at most  $2(q+q_d)^2/2^{128}$ ; absorbing the constant into the
690 stated bound yields
691  $\text{Adv}^{\text{IND}\text{-}\text{CCA}}_{\text{NAPQES}}(\mathcal{A})$ 
    \leq \text{Adv}^{\text{PRF}}_{\mathcal{B}_1}
        + \text{Adv}^{\text{PRF}}_{\mathcal{B}_2}
        + (q+q_d)^2/2^{128},
692 matching the statement.
693
694
695
    
```

CVF-16: openCategory: **documentation**

The PRF assumption on HMAC-SHA256 is used throughout but never formally defined

The quantity $\text{Adv}_{\text{HMAC-SHA256}}^{\text{PRF}}$ appears in every theorem bound, and the central game hop replaces $F(\text{kb}, \cdot)$ with a truly random function on the strength of it. Yet the paper never states the PRF assumption formally:

- There is no Definition of PRF advantage (no game, no distinguisher, no advantage expression) that Adv^{PRF} refers to.
- There is no explicit Assumption block asserting "HMAC-SHA256 is a PRF" that the theorems can cite.
- Critically, there is no key model. The key kb is the serialized prime-tuple (5 bytes per element), a structured, non-uniform value with only $\approx 2^{196}$ bits of effective entropy over a highly non-uniform distribution. The textbook Bellare HMAC-is-a-PRF result is stated for uniformly random keys, and its reduction to the compression-function assumptions does not transfer verbatim to a structured key drawn from this prime-tuple distribution.

Because the PRF term is invoked without a definition, an explicit assumption, or a justified key model, the security proofs are informal at their most load-bearing step: they silently assume PRF security of HMAC-SHA256 under a non-uniform key for which the standard result does not directly apply. Depending on the true distribution of kb , the assumption may not hold, in which case the bounds are not merely informal but potentially incorrect.

Recommendation

Add a formal PRF-advantage Definition (distinguishing game, distinguisher class, and the advantage expression $\text{Adv}_{\text{HMAC-SHA256}}^{\text{PRF}}$) and an explicit Assumption block before Theorem 1 stating that HMAC-SHA256 is a PRF, then reference every theorem bound to it. Address the key model directly: either argue that HMAC-SHA256 remains a PRF when keyed by the non-uniform prime-tuple serialization kb (e.g. via a randomness-extraction or key-derivation step that maps kb to a uniform key), or flag the non-uniform-key PRF security as an explicit additional assumption of the scheme. Do not invoke the standard uniform-key HMAC-PRF result without this justification.

Client Comment**/docs/napseq-eprint-preprint.tex**

```

61 IND-CPA security under the PRF assumption on HMAC-SHA256, discuss the
101 standard PRF assumption on HMAC-SHA256 and show that the
334 derivations always query the PRF at distinct inputs.
338 Under the PRF assumption on HMAC-SHA256, the NAPQES block
    encryption scheme

```

/rust/src/lib.rs

```

100 fn key_bytes(key: &[u64]) -> Vec<u8> {
104     out.extend_from_slice(&b[3..8]); // low 5 bytes, big-endian
110     let mut mac = HmacSha256::new_from_slice(kb).expect("hmac");

```

5.3. Moderate

CVF-17: open

Category: **readability**

varint is defined only as "unsigned LEB128 encoding", which is itself never defined

The notation $\text{varint}(x)$ is defined as "unsigned LEB128 encoding of non-negative integer x ", but LEB128 is neither defined nor given a normative reference anywhere in the paper. As written, the wire format cannot be reproduced from the specification alone: the byte layout, the continuation-bit convention, and whether a canonical (minimal-length) encoding is required are all left implicit. Beyond being undefined, variable-length LEB128 decoding is error-prone in ways that have security relevance for a decoder pipeline (overlong/non-canonical encodings, continuation-bit handling, unbounded length); the concrete decoding hazards are reported as a separate finding.

Recommendation

Define the unsigned LEB128 encoding explicitly — 7 data bits per byte, high bit as continuation flag, little-endian group order — or cite a normative reference. State that only canonical minimal-length encodings are valid and specify that non-canonical/overlong encodings and inputs exceeding the maximum permitted integer width are rejected during decoding.

Client Comment

/docs/napseq-eprint-preprint.tex

```

161 \item  $\text{varint}(x)$  denotes unsigned LEB128 encoding of
    non-negative integer  $x$ .

```

CVF-18: openCategory: **documentation**

Padding-codepoint derivation names domain 0x06 but omits the concrete formula

The plaintext padding defines **pad** as a $(B - n)$ -codepoint sequence "with each codepoint HMAC-derived via domain `\texttt{0x06}` in $[32, 126]$ ", but the concrete derivation formula is missing. Table 1 lists domain `\texttt{0x06}` as "Padding codepoint" with context $\text{be4}(\text{pad_idx})$ and output range $[32, 126]$, yet the paper never states how the 32-byte HMAC output of $\text{Derive}_{0x06}(\text{kb}, N, \text{be4}(\text{pad_idx}))$ is reduced to a single codepoint in $[32, 126]$: which bytes are taken and what mapping/reduction is applied. Without the explicit formula the padding is not reproducible from the specification, and the unstated mapping into a 95-value range ($[32, 126]$) is precisely the kind of reduction that introduces modulo bias when done naively.

Recommendation

State the explicit padding-codepoint formula: specify which bytes of $\text{Derive}_{0x06}(\text{kb}, N, \text{be4}(\text{pad_idx}))$ are consumed and the exact mapping into $[32, 126]$ (e.g. $32 + (x \bmod 95)$), consistent with the other domain derivations. Address the resulting bias in mapping to a 95-value range — either document that the bias is negligible for the chosen input width or use rejection sampling — so that all conforming implementations produce identical padding.

Client Comment**/docs/napseq-eprint-preprint.tex**

```
224 where \mathbf{pad} is a  $(B - n)$ -codepoint sequence with each
    codepoint
225 HMAC-derived via domain \texttt{0x06} in  $[32, 126]$ . The total padded
```

CVF-19: openCategory: **documentation**

Wire Format (Version 6): the ciphertext component B-tilde is used without a formal definition

The version-6 wire format is given as $C = N \| \widetilde{B} \| T$, but $\widetilde{B}$ is never formally defined. It is only described in prose as "the domain-`\texttt{0x07}` XOR-masked LEB128 token blob", which relies on terms whose precise construction is pinned down (if at all) elsewhere and gives no defining equation. In particular, the exact relation $\widetilde{B} = \text{LEB128}(\text{tokens}) \oplus \text{keystream}_{0x07}(N)$, the order of operations (encode-then-mask), the keystream length and truncation rule, and the token vector that feeds the encoding are not stated at the

point where $\widetilde{B}$ is introduced. As a result the wire format cannot be reproduced from this section alone.

Recommendation

Add an explicit defining equation for $\widetilde{B}$ in the wire-format section — e.g. state that $\widetilde{B} = \text{LEB128}(T_1, \dots, T_m) \oplus \text{ks}_{0x07}(N)$ truncated to the LEB128 blob length, with forward references to the token-construction and domain-`\texttt{0x07}` keystream definitions — so that $C = N \parallel \widetilde{B} \parallel T$ is fully and unambiguously specified in one place.

Client Comment

/docs/napseq-eprint-preprint.tex

```

257 C = N \parallel \widetilde{B} \parallel T,
259 where $N$ is a 16-byte random nonce, $\widetilde{B}$ is the
    domain-\texttt{0x07}
260 XOR-masked LEB128 token blob, and

```

CVF-20: open

Category: **architecture**

INT-CTXT Case 2 split ignores the AAD, and freshness is defined on the ciphertext only, not (c, aad)

Case 2 ($N^* = N_i$) splits into $\widetilde{B}^* \neq \widetilde{B}_i$ versus $\widetilde{B}^* = \widetilde{B}_i \wedge T^* \neq T_i$, and in the second sub-case claims "the tag input is identical to x_i ." That holds only if $\text{aad}^* = \text{aad}_i$, which is never assumed. If $\text{aad}^* \neq \text{aad}_i$ while $\widetilde{B}^* = \widetilde{B}_i$, then $x^* \neq x_i$ and the sub-case must route to the Case 1 (fresh-input) argument rather than being dismissed as unable to contribute. The correct split is on $x^* = x_i$ versus $x^* \neq x_i$, using injectivity of the length-prefixed encoding. Relatedly, freshness is defined as $c^* \notin \{c_1, \dots, c_q\}$ only; standard AEAD INT-CTXT treats the pair (c, aad) as the object of freshness, so replaying c_i with a new $\text{aad}^* \neq \text{aad}_i$ is a legitimate forgery attempt that this definition silently excludes rather than proving infeasible.

Recommendation

Re-split Case 2 on $x^* = x_i$ versus $x^* \neq x_i$, invoking injectivity of the encoding $0x03\parallel\text{be4}(|\text{aad}|)\parallel\text{aad}\parallel N\parallel\widetilde{B}$ and routing the fresh-input sub-case to the Case 1 argument. Define INT-CTXT freshness over the pair (c^*, aad^*) , and explicitly prove infeasibility of the replay-with-new-AAD attempt (same c_i , different aad^*).

Client Comment

/docs/napseq-eprint-preprint.tex

```

540 valid ciphertext: a tuple  $(c^*, \text{aad}^*)$  with
541  $c^* \notin \{c_1, \dots, c_q\}$  (the outputs of the  $q$  encryption
    oracle
542 calls) such that  $\text{NAPQES}.\text{Dec}(\mathbf{k}, c^*, \text{aad}^*) \neq \text{bot}$ .
543 Since  $c^* \neq c_i = (N_i, \widetilde{B}_i, T_i)$ , either
544  $\widetilde{B}^* \neq \widetilde{B}_i$ , in which case  $x^* \neq x_i$ 
    and
545 the argument from Case~1 applies; or  $\widetilde{B}^* = \widetilde{B}_i$ 
546 and  $T^* \neq T_i$ , but then  $c^*$  cannot be a valid ciphertext (the
    tag
547 input is identical to  $x_i$  yet the tag differs from the correct
    value),
548 so this sub-case cannot contribute to  $\mathcal{A}$ 's success.

```

CVF-21: openCategory: **architecture**

IND-CCA proof invokes Theorem 2 (INT-CTXT) in a decryption-oracle, $q+q_d$ regime it never proves

Theorem 2 (INT-CTXT) grants $\mathcal{A}$ only q encryption queries and, implicitly, a single forgery attempt, and its bound is stated purely in terms of q . The IND-CCA proof, however, invokes it via the Bellare–Namprempre formulation that grants the INT-CTXT adversary both an encryption oracle and a decryption oracle, and applies "Theorem 2 (INT-CTXT bound) with $q + q_d$ total oracle queries" — a multi-verification-query variant (with q_d decryption/verification attempts) that Theorem 2 never proves. With q_d verification attempts, the ideal-world guessing term (see the separate missing- 2^{-256} finding) also grows to $q_d/2^{256}$. As stated, the IND-CCA bound rests on an INT-CTXT statement strictly stronger than the one actually established.

Two further gaps compound this at the reduction step: (i) the $q^2/2^{128}$ term in Theorem 2 was a collision bound over encryption-generated nonces, but decryption queries generate no nonces, so substituting $q + q_d$ into that parameter has no basis in Theorem 2's proof; and (ii) the forger $\mathcal{A}'$ must itself produce the challenge ciphertext c^* (by choosing

b and calling the encryption oracle on m_b) — that is $q + 1$ encryption queries — yet the simulation description omits the challenge phase entirely while asserting, rather than showing, that " $\mathcal{A}'$'s simulation is perfect".

Recommendation

Prove INT-CTXT in the multi-verification-query formulation (encryption and decryption oracles, q encryption and q_d verification queries) with a bound that includes the $q_d/2^{256}$ guessing term, or restate the IND-CCA composition so it relies only on the single-forgery INT-CTXT actually proved. Count the challenge encryption ($q + 1$ queries) explicitly, justify the parameter used in place of Theorem 2's encryption-nonce collision term, and demonstrate (not assert) that $\mathcal{A}'$'s simulation is perfect across both phases.

Client Comment

/docs/napseq-eprint-preprint.tex

```

527 Specifically, for any PPT adversary  $\mathcal{A}$  making at most  $q$ 
528 encryption queries there exists a PRF adversary  $\mathcal{B}_2$  with
632 \]
633
634 We bound  $\Pr[D]$  by constructing an INT-CTXT forger  $\mathcal{A}'$ 
    (using
640 \item  $\mathcal{A}'$  simulates game  $H_0$  for  $\mathcal{A}$ ,
    forwarding
641 every one of  $\mathcal{A}$ 's encryption queries to
 $\hat{\mathcal{E}}$ 
642 and every decryption query to  $\hat{\mathcal{D}}$ .
643 \item Whenever  $\hat{\mathcal{D}}$  returns a non- $\bot$  result on a
644 fresh ciphertext  $(c, \text{aad})$  (i.e., one not previously output
    by
645  $\hat{\mathcal{E}}$ ), event  $D$  has occurred;  $\mathcal{A}'$ 
    wins the
646 INT-CTXT experiment.
648  $\mathcal{A}'$ 's simulation is perfect (it uses the real oracles
    throughout),
649 so  $\Pr[\mathcal{A}' \text{ wins INT-CTXT}] = \Pr[D]$ , giving:
656 +  $\frac{(q+q_d)^2}{2^{128}}$ ,
659  $q+q_d$  total oracle queries.
```

CVF-22: openCategory: **architecture**

IND-CCA proof: event D is restricted to the query phase, leaving post-challenge forgeries uncovered

Game H_0 explicitly allows decryption queries after the challenge (" $\mathcal{A}$ may call $\mathcal{D}$ on any $c \neq c^*$ "), but the bad event D is defined as a fresh valid submission "during the query phase" only. A fresh valid ciphertext submitted in the post-challenge phase makes H_0 and H_1 diverge yet is not counted by D , so the bound $|\Pr[H_0] - \Pr[H_1]| \leq \Pr[D]$ once again fails to cover all distinguishing executions.

Recommendation

Define D over all decryption queries in both the pre- and post-challenge phases (any fresh valid (c, aad) submitted to $\mathcal{D}$ at any point), and ensure the INT-CTXT reduction bounding $\Pr[D]$ runs across both phases.

Client Comment**/docs/napseq-eprint-preprint.tex**

```

615 oracle  $\mathcal{E}$  and a decryption oracle  $\mathcal{D}$ . After
    receiving
616 the challenge ciphertext  $c^*$ ,  $\mathcal{A}$  may call  $\mathcal{D}$ 
    on
622 \paragraph{Transition  $H_0$  to  $H_1$ .}
623 Let  $D$  be the event that  $\mathcal{A}$  submits, during the query
    phase, a

```

CVF-23: openCategory: **architecture**

IND-CCA proof: freshness-set mismatch and nonstandard c -only freshness weaken the notion

The bad event D uses the freshness set $\{c_1, \dots, c_q\}$, but from the INT-CTXT challenger's view the challenge ciphertext c^* is also an $\hat{\mathcal{E}}$ output, so the forger $\mathcal{A}'$ wins only on ciphertexts fresh with respect to $\{c_1, \dots, c_q\} \cup \{c^*\}$; the proof never reconciles these sets. This is harmless here only because H_0 forbids querying $c = c^*$ — but that restriction is itself nonstandard for AEAD: the usual IND-CCA game forbids only the pair (c^*, aad^*) and permits $\mathcal{D}(c^*, aad \neq aad^*)$. The game as written therefore silently weakens the adversary, consistent with the c -only (rather than (c, aad)) freshness already flagged for the

INT-CTXT proof, so the composed result establishes a weaker-than-standard IND-CCA notion.

Recommendation

Reconcile the freshness set used by D with the INT-CTXT win condition (include c^*), and define IND-CCA freshness over the pair (c^*, aad^*) — permitting $\mathcal{D}(c^*, aad \neq aad^*)$ — so the proven notion is standard AEAD IND-CCA rather than a c -only weakening.

Client Comment

/docs/napseq-eprint-preprint.tex

```

616 the challenge ciphertext  $c^*$ ,  $\mathcal{A}$  may call  $\mathcal{D}$ 
    on
624 \emph{fresh valid} ciphertext to  $\mathcal{D}$ —i.e., a tuple
643 \item Whenever  $\hat{\mathcal{D}}$  returns a non-\bot result on a
644     fresh ciphertext  $(c, aad)$  (i.e., one not previously output
    by
645      $\hat{\mathcal{E}}$ ), event  $D$  has occurred;  $\mathcal{A}'$ 
    wins the

```

CVF-24: open

Category: **readability**

Grover post-quantum key-search statement is cryptic and potentially misleading

The paper states that "Grover's algorithm halves the exponent, giving a post-quantum key-search complexity of $\approx 2^{98}$, well above 128-bit thresholds," presented as a bare figure with no operational caveats. This is cryptic and can mislead a reader in two ways. First, 2^{98} read as an ordinary attack cost invites the (false) reading that a 2^{98} -work quantum attack is a real, achievable threat. Second, and more importantly, Grover's algorithm requires $\approx 2^{n/2}$ essentially sequential oracle queries and is well known to be non-parallelizable in any useful sense: Zalka's analysis and the BBBV optimality result show that distributing the search over S machines yields only a $\sqrt{S}$ speedup, so the wall-clock depth stays $\approx 2^{n/2}/\sqrt{S}$ and the total work is unchanged. Adding hardware therefore does not bring these numbers into feasible range. Stated without this context, the 2^{98} figure looks like a parallelizable, hardware-scalable cost rather than the sequential-depth lower bound it actually is.

Recommendation

Rephrase the Grover statement to make its operational meaning explicit: Grover's algorithm needs $\approx 2^{n/2}$ sequential oracle queries and is essentially non-parallelizable (cite Zalka and the BBBV optimality bound — S machines give only a $\sqrt{S}$ speedup, wall-clock depth stays $\approx 2^{n/2}/\sqrt{S}$, total work unchanged). Present both the 2^{128} SHA-256 preimage bound and the 2^{98} key-space bound as sequential-depth quantities far beyond feasibility that cannot be reached by adding hardware, and note that at these output lengths Grover is largely irrelevant to practical security.

Client Comment

/docs/napseq-eprint-preprint.tex

```

91 quantum speedup beyond Grover's  $\sqrt{N}$  search, which reduces
    the
92 effective security level from 256~bits to approximately 128~bits
    while
705 Grover's algorithm~\cite{grover96} provides a quadratic speedup for
    unstructured
706 search. Applied to SHA-256 preimage search, it reduces the effective
    security
717  $\approx 2^{196}$ . Grover's algorithm halves the exponent, giving a
718 post-quantum key-search complexity of  $\approx 2^{98}$ , well above

```

5.4. Minor

CVF-25: open

Category: **documentation**

Wire-format v6 components listed in the contributions are undefined at their point of use

The contributions list promises a specification of NAPQES wire format v6 “including the key serialisation, HMAC derivation schedule, token construction, padding scheme, and authentication tag.” Each of these terms is introduced here without a definition or a forward reference to where it is defined, so a reader cannot, from this list, locate the precise meaning of the promised components. “Key serialisation”, “HMAC derivation schedule”, “token construction”, “padding scheme”, and “authentication tag” are all specialised terms whose exact content the paper claims to define but does not pin down at this point.

Recommendation

Add a forward reference (subsection label) next to each named component so the reader can jump to its definition, or briefly define each term at first mention. This makes the contributions claim verifiable and gives each promised artifact a single authoritative definition site.

Client Comment**/docs/napseq-eprint-preprint.tex**

```

97 \item We specify NAPQES wire format v6
    (Section~\ref{sec:construction}),
98     including the key serialisation, HMAC derivation schedule,
99     token construction, padding scheme, and authentication tag.
```

CVF-26: openCategory: **behavior**

Prime-tuple sampling procedure is unspecified: uniform over primes or over tuples, and how

The key is defined as "an ordered tuple of K distinct primes drawn uniformly from $\mathcal{P}$ using a cryptographically secure DRBG", but the sampling procedure is not specified. It is ambiguous what "uniformly" quantifies over: individual primes drawn uniformly from $\mathcal{P}$ (with rejection of duplicates) versus the ordered K -tuple drawn uniformly from the set of all distinct ordered K -tuples. These are different distributions and support different key-space arguments. The paper also does not state how uniform sampling from $\mathcal{P}$ is realised from DRBG output — e.g. indexing into a precomputed prime table versus sampling integers and primality-testing, and whether rejection sampling is used to avoid modulo bias — which is exactly where uniformity is easy to lose.

Recommendation

Specify the exact sampling algorithm: state whether uniformity is over primes or over ordered distinct tuples, the deduplication/rejection strategy, and how DRBG bytes are mapped to an element of $\mathcal{P}$ without modulo bias (e.g. rejection sampling over an index in $[0, |\mathcal{P}|)$). Make explicit which distribution the 2^{196} key-space estimate assumes.

Client Comment**/docs/napseq-eprint-preprint.tex**

```

166
167 A NAPQES key  $\mathbf{k} = (k_1, k_2, \ldots, k_K)$  is an ordered
    tuple
168 of  $K$  distinct primes drawn uniformly from  $\mathcal{P}$  using a

```

CVF-27: openCategory: **documentation**

Domain-separated derivation formula does not define kb and N , nor their lengths

The domain-separated derivation is defined as $\text{Derive}_d(kb, N, ctx) = \text{HMAC}(kb, N \| d \| ctx)$, but kb and N are introduced here without definition. kb is presumably the serialised key material key_bytes and N a nonce, but neither is stated, and crucially their byte lengths are unspecified. Because the HMAC message is the concatenation $N \| d \| ctx$ with d a single byte and ctx variable across domains, an unspecified or variable length for N can make the boundary between N , d , and ctx ambiguous, which undermines the domain separation the whole schedule relies on.

Recommendation

Define kb (e.g. $kb = key_bytes$) and N (the nonce) with their exact byte lengths at this point. State that N has a fixed length so that $N \| d \| ctx$ is unambiguously parseable; if N is variable-length, prepend an explicit length encoding so domain separation cannot be violated by boundary-shifting between fields.

Client Comment**/docs/napseq-eprint-preprint.tex**

```

184 \mathrm{Derive}_d(\mathsf{kb}, N, \mathsf{ctx}) =
    \mathrm{HMAC}(\mathsf{kb}, N \parallel d \parallel \mathsf{ctx}),

```

CVF-28: openCategory: **typing**

Numeric representation of the noise probability p is unspecified (fixed-point vs floating-point)

The noise probability is defined as $p = 0.75 + t \cdot 0.24$ with $t = \text{Derive}_{0x02}(\text{kb}, N, \varepsilon)[0 : 8]_{\text{be64}} / 2^{64}$. The data type and numeric representation of t and p are not specified: fixed-point, IEEE-754 double, or exact rational. Since p drives a per-position Bernoulli decision (domain 0x00, "noise position"), any divergence in how p is represented, rounded, and compared across implementations or platforms would change which positions are classified as noise, breaking determinism and cross-implementation interoperability of encryption and decryption.

Recommendation

Specify the exact numeric type and rounding for t and p , preferably a fixed-point representation derived directly from the 64-bit integer (avoiding floating point entirely), and define the exact integer comparison used to derive the Bernoulli noise decision, so that all conforming implementations agree bit-for-bit on which positions are noise.

Client Comment**/docs/napseq-eprint-preprint.tex**

```

212 \[
213   t = \frac{\mathrm{Derive}_{0x02}(\mathsf{kb}, N,
      \varepsilon)[0:8]_{\mathrm{be64}}}{2^{64}},
214   \quad
215   p = 0.75 + t \cdot 0.24.
216 \]
```

CVF-29: openCategory: **readability**

"codepoint" is used as the plaintext length unit but is never defined

The padding section sets $n = |P|$ as the "plaintext length in codepoints", and codepoints are then used throughout (padded length $B + 2$, HMAC-derived padding in $[32, 126]$, and the block-mode cap). However "codepoint" is never defined: it is unclear whether it denotes a single byte, a UTF-8/UTF-16 code unit, or a Unicode scalar value, and which encoding maps the plaintext to the $[32, 126] / [32, 127]$ character ranges used in the token and padding construction. This ambiguity propagates into the padded-length computation and the 16-bit length cap, so $|P|$ is not well-defined as specified.

Recommendation

Define “codepoint” precisely — for example a single byte in a stated range, or a Unicode scalar value under an explicitly named encoding — and specify the encoding that maps plaintext to the character ranges referenced by the token and padding construction, so that $n = |P|$ and all derived lengths are unambiguous.

Client Comment

/docs/napseq-eprint-preprint.tex

220 | Let $n = \lvert P \rvert$ (plaintext length in codepoints). Define:

CVF-30: open

Category: **behavior**

AAD length prefix is only 32 bits, so AAD framing and binding are ambiguous for very large AAD

The authentication tag is $T = \text{HMAC}(\text{kb}, 0x03 \parallel \text{be4}(|A|) \parallel A \parallel N \parallel B)$, where be4 is a 4-byte length field. AADs of length L and $L + 2^{32}$ therefore share the same length prefix, defeating the unambiguous framing the paper relies on (“the 4-byte length prefix prevents length-extension mixing between different AAD lengths”). The reference implementation encodes the length as `\texttt{(aad.len() as u32).to_be_bytes()}`, which silently truncates the `\texttt{usize}` to 32 bits, so the 32-bit ceiling is shared by specification and code. Exploitation requires at least 4 GiB of AAD, so it is theoretical, but the framing guarantee does not hold at that scale and the truncation is silent rather than rejected.

Recommendation

Use a length field wide enough to cover the supported AAD range (e.g. an 8-byte be8 length prefix), or explicitly document and enforce a maximum AAD length below 2^{32} and reject longer inputs. In the code, replace the `\texttt{as u32}` cast with a checked conversion that errors on overflow instead of silently truncating.

Client Comment

/docs/napseq-eprint-preprint.tex

```

197 \texttt{0x03} & Auth tag (block) & $\mathrm{be4}(\llvert aad \rrvert)
    \ll aad \ll \mathsf{payload}$ & 32 bytes \ll
584 $T = \mathrm{HMAC}(\mathsf{kb}, \ll \texttt{0x03} \ll \mathrm{be4}(|A|)
    \ll A \ll N \ll B)$ .
585 The 4-byte length prefix prevents length-extension mixing between
    different

/rust/src/lib.rs
188     buf.extend_from_slice(&(aad.len() as u32).to_be_bytes());

```

CVF-31: openCategory: **behavior**

Unbounded ciphertext expansion — geometric noise with no hard cap enables resource amplification

Noise emission is geometric with probability $p \in [0.75, 0.99)$ (derived as $p = 0.75 + 0.24 t$), and the emission loop has no hard cap on the number of noise tokens per codepoint. For $p \rightarrow 0.99$ the expected expansion is $\approx 99\times$, so a 64 KiB plaintext can produce tens of megabytes, and worst-case per-codepoint iteration counts are large — a resource-amplification / denial-of-service vector. Table 4 and CAV-004 characterise the expansion as “4–20 $\times$ ”, which understates the tail near the 0.99 ceiling. The reference implementation’s noise loop (`\texttt{is_noise_pos}`) has no iteration cap, and `\texttt{derive_noise_p}` is bounded only at 0.99.

Recommendation

Impose a hard per-codepoint and per-message cap on the number of noise tokens, and document the true worst-case expansion at the 0.99 ceiling rather than the “4–20 $\times$ ” typical figure. Consider lowering the maximum noise probability or bounding total ciphertext length, and abort encryption when the cap is exceeded so a chosen input cannot be amplified without bound.

Client Comment

/docs/napseq-eprint-preprint.tex

```

212 \[
213   t = \frac{\mathrm{Derive}_{\texttt{0x02}}(\mathsf{kb}, N,
    \mathsf{varepsilon})[0:8]_{\mathrm{be64}}}{2^{64}},
214   \quad
215   p = 0.75 + t \cdot 0.24.

```

```

216 \]
745 The primary trade-off is ciphertext expansion: NAPQES's
    noise-injection
746 mechanism produces $4\text{--}20\times$ ciphertext expansion
    depending on the
747 derived noise probability, compared to no expansion for AES-GCM and
874 \item[CAV-004 – Ciphertext expansion (informational).] Noise
    injection
875     produces $4\text{--}20\times$ ciphertext expansion relative to
876     plaintext size. This is a deliberate confidentiality
    feature, not a
877     bug; it is documented in all datasheets. No fix is planned.

/rust/src/lib.rs

182     0.75 + t * (0.99 - 0.75)
265     loop {

```

CVF-32: openCategory: **documentation**

IND-CCA proof: minor wording and citation issues ("computationally" vs perfectly equivalent; B&N numbering)

Two minor issues in the IND-CCA proof. First, " H_1 ... is therefore computationally equivalent to the IND-CPA experiment" understates the step: with the decryption oracle fixed to $\perp$ (trivially simulatable), H_1 is perfectly (information-theoretically) equivalent to the IND-CPA experiment, not merely computationally, and the current wording muddies the reduction. The step also silently assumes $\mathcal{A}$'s query budget matches Theorem 1's (it does — q encryption queries), which should be stated. Second, the Bellare–Namprepre citation attributes the composition result and the INT-CTXT-with-oracles formulation to "Theorem 3" and "Definition 2" of the reference; these numbers differ across the proceedings and ePrint versions of B&N 2000 and should be verified against the exact edition cited.

Recommendation

Replace "computationally equivalent" with "perfectly (information-theoretically) equivalent" and state the query-budget correspondence with Theorem 1 explicitly. Verify and correct the Bellare–Namprepre theorem/definition numbers against the specific cited version.

Client Comment

/docs/napseq-eprint-preprint.tex

```

608 We apply the composition theorem of Bellare and
634 We bound  $\Pr[D]$  by constructing an INT-CTXT forger  $\mathcal{A}'$ 
    (using
664 computationally equivalent to the IND-CPA experiment, giving:
675 where Theorem~1 is applied to the IND-CPA adversary  $\mathcal{A}'$ .

```

6. Conclusion

The review of the NAPQES ePrint preprint identified thirty-two issues: two of critical severity, fourteen of high severity, eight of moderate severity, and eight of low severity. The findings indicate that, as specified, the scheme does not achieve the confidentiality goal it claims and that its security proofs contain substantive gaps; the specification also suffers from numerous consistency and completeness problems that would prevent an interoperable, correct implementation.

The two critical findings concern confidentiality and specification consistency at the wire-format level. The padding scheme leaks the plaintext length bucket through the ciphertext byte length, so the ciphertext length is correlated with the plaintext content, contradicting the IND-CPA claim; and the domain-separation layout is described inconsistently across Table 1, the security proof, and the derivation formulas, so no single interpretation is both internally consistent and matched to the tag construction Derive_{0x03} .

The high-severity findings are dominated by defects in the security analysis and architecture. They include a length-channel key-recovery path under nonce reuse that contradicts the stated misuse posture; the reliance of all confidentiality on the domain- $0x07$ keystream while the prime-token layer contributes no proven secrecy; the absence of a formal AEAD algorithm triple; incomplete token-emission pseudocode; and a series of errors in the IND-CPA, INT-CTXT, and IND-CCA theorems, including bounds that ignore key entropy, an omitted tag-guessing term, an adversarially controlled nonce-collision probability, unjustified freshness and cross-group HMAC arguments, reductions that cannot simulate the encryption oracle, a false “identical-until-bad” step, and an unstated PRF assumption on HMAC-SHA256.

The moderate-severity findings continue the pattern in the proofs and specification: an under-specified varint encoding, an INT-CTXT case split that ignores the associated data, an IND-CCA proof that misapplies the INT-CTXT theorem in a decryption-oracle setting, freshness-set and event-scope mismatches, and a cryptic Grover-based post-quantum key-search statement. The low-severity findings are predominantly documentation and robustness gaps: undefined wire-format components, an unspecified prime-tuple sampling procedure, undefined symbols in derivation formulas, an ambiguous noise-probability representation, a narrow 32-bit AAD length prefix, and unbounded ciphertext expansion from the geometric noise process.

All issues remain open at the time of this draft. We recommend that the confidentiality goal be restated against a precise AEAD definition, that the padding and length-hiding behaviour be reconciled with that goal, that the domain-separation schedule be made unambiguous and self-consistent, and that each theorem be re-proved with fully specified games, oracles, and bounds before the scheme is considered for deployment.

Client Comment



ABDK

Consulting

About Us

Established in 2016, is a leading service provider in the space of blockchain development and audit. It has contributed to numerous blockchain projects, and coauthored some widely known blockchain primitives like Poseidon hash function.

The ABDK Audit Team, led by Mikhail Vladimirov and Dmitry Khovratovich, has conducted over 300 audits of blockchain projects in Solidity, Rust, Circom, C++, Javascript, and other languages.

Contacts

Email

info@abdkconsulting.com

Website

abdk.consulting

X (former Twitter)

[@ABDKConsulting](https://twitter.com/ABDKConsulting)

LinkedIn

company/abdk-consulting